

GLiNER vs spaCy

A detailed technical analysis of Named Entity Recognition approaches

1. The Core Paradigm Shift

spaCy treats NER as a **token classification** problem — a trained head predicts one of a fixed set of labels (PER, ORG, LOC, etc.) for each token. This locks entity types at training time. GLiNER reframes the problem entirely: NER becomes a **matching problem** where entity type embeddings are compared to textual span representations in a shared latent space. Entity types are expressed in plain natural language, and a Bidirectional Language Model (DeBERTa-v3) jointly encodes both labels and input text in a single forward pass.

2. Key Advantages of GLiNER

Advantage	GLiNER	spaCy
Zero-shot generalization	Any entity type at runtime (natural language label)	Fixed at training time (PER, ORG, LOC, etc.)
Nested entity support	Native — overlapping spans supported out of the box	Not supported by default
Multilingual	Cross-lingual transfer built-in; beats ChatGPT in 8/10 languages	Separate language-specific models required
Domain adaptation	Describe new entity types in natural language — no labeled data	Needs labeled data + fine-tuning for new domains
Inference efficiency	Parallel span scoring — $O(n \log n)$; <500M params, CPU-only	Very fast Cython pipeline but fixed entity vocabulary
Privacy / local deployment	Fully local — no API calls; ideal for PII redaction	Also local, but less flexible for custom PII types
Natural language labels	"medication dosage", "Indian city" — any description works	Fixed class IDs only

3. GLiNER Architecture

GLiNER has three main components that work together to enable arbitrary entity type extraction:

1	Input Encoding	Entity type labels and input text are concatenated into a single sequence. Each entity type is prefixed with a special [ENT] token. Both are passed through DeBERTa-v3 together, giving the model joint context over labels and text in one forward pass.
2	Span Representation Module	All possible token spans (contiguous sub-sequences up to a max length) are enumerated. For each span (i, j), a span embedding is computed by combining BiLM token outputs through a two-layer FFN — this captures multi-word entities like 'New York City' as a single vector.
3	Entity Representation Module	Each entity type embedding (the [ENT] token output) is projected via a separate FeedForward network into the same shared latent space as the span embeddings.

4	Matching & Scoring	A dot product is computed between every span embedding and every entity embedding, followed by sigmoid activation. This gives a score in (0,1) for every (span, entity_type) pair — the key design enabling arbitrary entity types without retraining.
5	Greedy Span Decoding	After thresholding scores, a greedy algorithm selects final spans. Flat NER: highest-scoring non-overlapping spans. Nested NER: overlapping spans allowed (no partial overlaps). Complexity is $O(n \log n)$ — far faster than autoregressive LLM decoding.

4. Training Pipeline

Step 1: Dataset — Pile-NER

50,000 texts sampled from The Pile corpus, annotated by ChatGPT without predefined type constraints. Result: 44,889 passages, 240,000 entity spans, 13,000 distinct entity types. This diversity is the foundation of GLiNER's generalization.

Step 2: Backbone — DeBERTa-v3

Chosen for empirically strong bidirectional representations. Non-pretrained layers (FFN, span representation head) use hidden width 768 and dropout 0.4 to regularize training.

Step 3: Loss — Binary Cross-Entropy with in-batch negatives

Positive pairs: spans labeled with the entity type in the training sentence. Negative pairs: entity types randomly sampled from other examples in the same batch. Focal loss weighting (0.75 positive / 0.25 negative) handles class imbalance.

Step 4: Optimizer & Schedule

AdamW with differential learning rates: $1e-5$ for pretrained transformer layers, $5e-5$ for new FFN heads. 30,000 steps total with 10% linear warmup then cosine decay. Max sequence: 768 tokens. Max labels per example: 30.

Step 5: Zero-shot Transfer

Because the model learns to match arbitrary natural language entity descriptors to spans — not a fixed classification head — it generalizes to unseen entity types and languages by embedding them into the same shared latent space.

5. Performance Benchmarks

German PII redaction (F1 score) — Nov 2024 benchmark (Nils Durner):

Model	F1 Score	Notes
GLiNER	94.92%	Best overall accuracy
Gemma-2-9B	93.22%	Close competitor (8B+ params)
spaCy-GLiNER (hybrid)	91.53%	GLiNER integrated into spaCy pipeline
Llama-3.1-8B-Instruct	86.44%	Over-redacts in some cases
Llama-3.2-3B-Instruct	81.36%	Smaller variant
spaCy de_core_news_lg	77.97%	Standard spaCy German model

6. Limitations of GLiNER

Noisy / informal text	Struggles with social media slang, abbreviations, and non-standard grammar. spaCy's rule-based matchers handle these better.
Complex multi-token financial entities	Stock symbols like 'NASDAQ: AAPL' and multi-clause spans frequently missed or truncated.
Ecosystem maturity	spaCy has vastly more integrations, pretrained pipelines, and battle-tested deployments. GLiNER's ecosystem is newer (GLiNER2, GLiREL, GLiClass are emerging).
Low-resource languages	Languages not represented in DeBERTa-v3 pretraining or Pile-NER see degraded performance.
Span enumeration at scale	Enumerating all possible spans for very long documents can be memory-heavy without chunking.
Structured output consistency	Without fine-tuning, span boundaries can be inconsistent on edge cases.

7. When to Choose GLiNER vs spaCy

Use GLiNER when...	Use spaCy when...
Entity types are dynamic or domain-specific (e.g. medical terms, Indian organizations)	Entity vocabulary is fixed and well-defined (standard PER, ORG, LOC suffice)
You need zero-shot extraction of novel entities without retraining	You need maximum inference throughput on high-volume, fixed-label pipelines
Nested entity detection is required (e.g. entity within entity)	Deep spaCy ecosystem integration needed (displacy, matchers, custom components)
PII redaction in privacy-sensitive environments (healthcare, legal, finance)	Noisy/informal text is the primary input (social media, chat logs)
Multilingual extraction without separate language models	You need battle-tested production reliability with extensive community support

Sources: Zaratiana et al. (2024) — GLiNER paper (arXiv:2311.08526) · Nils Durner NER benchmark (Nov 2024) · GLiNER2 arXiv:2507.18546 · CodeCut GLiNER vs spaCy analysis (Feb 2026)